

API Contract

Status: draft - **Freeze at hour 4.** After the freeze, this file changes *before* the code does - edit here, announce in the team channel, then implement.

This is the interface between one backend and five frontend people. It covers **every** feature in `feature_list.json`, not just the golden path, so nobody is blocked waiting for a decision.

Conventions

Base URL	VITE_API_BASE (the tunnel URL). No /api prefix.
Content type	application/json everywhere except file upload (multipart/form-data) and SSE.
Case	snake_case in all request and response bodies.
Auth	Authorization: Bearer <token> on everything except /health, /languages, /auth/signup, /auth/login.
Timestamps	ISO 8601, UTC, Z suffix. e.g. 2026-08-23T11:48:25Z
IDs	Integers, except the session token (opaque string).
Pagination	?limit= (default 50, max 200) and ?offset=. Responses wrap in { "items": [...], "total": n }.

Error envelope

Every non-2xx response, without exception:

```
{ "error": { "code": "insufficient_evidence", "message": "Human-readable.", "detail": {} } }
```

Code	Meaning
400 bad_request	Malformed input
401 unauthenticated	Missing / invalid / logged-out token
403 forbidden	Valid token, wrong role
404 not_found	
409 conflict	e.g. email already registered
422 validation_error	detail carries per-field messages
503 provider_unavailable	All LLM providers failed, including fallbacks

Roles

`student` - `teacher` - `admin`. A 403 is returned when the role does not match the route prefix. `/tutor/*` is student-only.

Shared objects

Referenced throughout. Defined once here.

User

```
{
```

```

    "id": 1,
    "email": "asha@example.edu",
    "full_name": "Asha R",
    "role": "student",
    "course_id": 3,
    "preferred_language": "en"
  }

```

Citation - rag-001, student-004

Produced by retrieval (rag-001) and attached to anything the tutor generates. **Never empty on an answered response** - that is the whole point of the build.

```

{
  "chunk_id": 812,
  "material_id": 4,
  "book_title": "Concepts of Physics, Vol 1",
  "page_no": 143,
  "chapter": "5. Newton's Laws of Motion",
  "snippet": "...the net force on a body equals..."
}

```

EvidenceReport

The alignment score (rag-002) and the refusal decision (rag-003) come from one object.

```

{
  "alignment_score": 0.82,
  "alignment_percent": 82,
  "top_similarity": 0.79,
  "threshold": 0.35,
  "sufficient": true,
  "reason": null
}

```

alignment_percent is what the UI renders. When **sufficient** is **false**, **reason** is a short string and the caller receives a refusal outcome instead of prose.

Gap

```

{
  "id": 21,
  "concept": "Vector components",
  "prerequisite_course": "Class 12 Physics",
  "detected_from": "diagnostic",
  "status": "open"
}

```

detected_from: diagnostic | syllabus_upload | practice. **status:** open | improving | closed.

TutorResponse - discriminated on outcome

Every tutor-generated response uses this shape. **Frontend must branch on `outcome`.**

```

{
  "outcome": "answered",
  "language": "en",
  "body": "A vector's components are...",
  "citations": [ /* Citation */ ],
  "evidence": { /* EvidenceReport */ }
}

```

```

{
  "outcome": "insufficient_evidence",
  "language": "en",
  "body": "I don't have approved course material covering this. I've flagged it for your teacher"
}

```

```
.",
  "citations": [],
  "evidence": { "sufficient": false, "alignment_percent": 11, "reason": "no_matching_material" }
,
  "uncertainty_flag_id": 55
}
```

```
{
  "outcome": "graded_work_refused",
  "language": "en",
  "body": "This looks like it's from a graded assignment, so I won't solve it. Here's how to approach it.",
  "hints": ["Start by resolving the force into components.", "..."],
  "citations": [ /* Citation */ ],
  "matched_assignment": { "material_id": 9, "title": "Problem Set 3" }
}
```

Misconception

```
{
  "id": 7,
  "label": "Treats velocity as implying a net force",
  "topic_id": 12,
  "problem_type": "newtons-second-law",
  "confirmed_count": 14
}
```

Practiceltem

```
{
  "id": 301,
  "prompt": "A 2 kg block slides at constant velocity...",
  "kind": "mcq",
  "options": ["...", "...", "...", "..."],
  "gap_id": 21,
  "citations": [ /* Citation */ ]
}
```

kind: mcq | numeric | short_text. **options** present only for mcq. **Correct answers are never sent to the client.**

System

Method	Path	Feature	Returns
GET	/health	infra-002	{ "status": "ok", "db": "ok" } - see note
GET	/meta/provider-status	infra-004	{ "active": "glm", "fallbacks_available": ["gemini", "groq"], "cache_enabled": true }
GET	/languages	i18n-001	{ "items": [{ "code": "en", "label": "English" }, { "code": "hi", "label": "???????" }] }

/meta/provider-status exists so the demo can prove on screen which provider is live and that the cache is on.

/health is always 200 while the process is up. If the database is unreachable it returns

{ "status": "degraded", "db": "<ErrorType>" } rather than a 5xx, so a dropped free-tier connection reads as "database down", not "app dead". Treat `status != "ok"` as a warning banner, never as a fatal error.

Auth - auth-001, auth-002

Method	Path	Body	Returns
POST	/auth/signup	{ email, password, full_name, role, course_id? }	201 { token, user }
POST	/auth/login	{ email, password }	200 { token, user }
POST	/auth/logout	-	204
GET	/auth/me	-	200 User
PATCH	/auth/me/preferences	{ preferred_language }	200 User

- `token` is an opaque UUID stored in a `sessions` table. **Not a JWT** - do not attempt to decode it client-side.
- Wrong password -> **401 unauthenticated**. Existing email on signup -> **409 conflict**.
- Logout invalidates the token server-side; the next call returns **401**.

Forgot password has no endpoint. auth-002 is UI-only by decision: the screen collects an email and shows confirmation copy. The frontend must not issue a network request. Do not add POST /auth/forgot-password - its absence is intentional.

Admin - admin-001, admin-002, admin-003, ingest-001

Structure (`admin-002`):

Method	Path	Notes
GET	/admin/departments	{ items: [{ id, name }] }
POST	/admin/departments	{ name }
GET	/admin/courses	{ items: [Course] }
POST	/admin/courses	{ code, title, department_id, prerequisite_course_ids: [] }
GET	/admin/courses/{course_id}	single Course

```
// Course
{ "id": 3, "code": "PH101", "title": "Mechanics", "department_id": 1,
  "prerequisite_courses": [ { "id": 1, "code": "PH000", "title": "Class 12 Physics" } ] }
```

Curriculum upload and versioning (`admin-001`):

Method	Path	Notes
POST	/admin/courses/{course_id}/materials	multipart: file, kind, title, chapter_map?
GET	/admin/courses/{course_id}/materials	?include_archived=false
GET	/admin/materials/{material_id}	single Material
POST	/admin/materials/{material_id}/archive	200 Material with status: "archived"
GET	/admin/materials/{material_id}/versions	{ items: [Material] }, newest first

```
// Material
{ "id": 4, "course_id": 3, "title": "Concepts of Physics, Vol 1", "kind": "textbook",
  "version": 2, "status": "active", "page_count": 412,
  "uploaded_by": "admin@example.edu", "uploaded_at": "2026-08-23T09:00:00Z",
  "ingest_status": "complete", "chunk_count": 1840 }
```

kind: syllabus | textbook | notes | assignment. status: active | archived.

kind: "assignment" is what powers the graded-work guardrail (rag-004). Material uploaded as assignment is retrievable for matching but never quoted as an answer.

Upload by a verified admin counts as approval for this build. There is no separate approval step for admin-uploaded material.

Ingestion (ingest-001):

Method	Path	Notes
POST	/admin/materials/{material_id}/ingest	202 { job_id } - parse + chunk + embed
GET	/admin/ingest-jobs/{job_id}	{ job_id, status, pages_done, pages_total, chunk_count, error? }

status: queued | running | complete | failed. Frontend polls this; there are no websockets anywhere in this build.

Audit log (admin-003):

Method	Path	Notes
GET	/admin/audit-log	?limit=&offset=&actor=&action=

```
{ "id": 88, "actor_email": "admin@example.edu", "action": "material.upload",
  "target": "material:4", "at": "2026-08-23T09:00:00Z", "detail": { "version": 2 } }
```

Actions: material.upload - material.archive - material.ingest - course.create - reteach.approve - sourced_content.approve - sourced_content.reject.

Student

Course scope - student-001

GET /student/course-summary

```
{
  "course": { "id": 3, "code": "PH101", "title": "Mechanics" },
  "books": [ { "material_id": 4, "title": "Concepts of Physics, Vol 1",
    "pages": "1-212", "chapters": ["1. Introduction", "..."] } ],
  "topics": [ { "id": 12, "name": "Newton's Laws" } ]
}
```

Prerequisite gap check - student-002

Method	Path	Notes
GET	/student/diagnostic	{ diagnostic_id, items: [Practiceltem] }
POST	/student/diagnostic/{diagnostic_id}/submit	{ answers: [{ item_id, answer }] }

POST	/student/syllabus-upload	multipart: file - alternative entry for incoming students
GET	/student/gaps	{ items: [Gap] }

Both submit paths return:

```
{ "gaps": [ /* Gap */ ], "message": "Found 3 prerequisite gaps." }
```

There is no score, percentage, or grade in this response, by design. The problem statement asks for a gap list, not a grade. Frontend must not compute one from answers.

Lesson for a gap - student-003, student-004, rag-002

GET /student/gaps/{gap_id}/lesson?language=en

Returns a **TutorResponse**. **outcome** is normally **answered**; **evidence.alignment_percent** is the badge on the card, and **citations[]** powers **Show Source** - each entry resolves to a real **book_title** + **page_no**.

Mastery - student-007

GET /student/mastery

```
{ "items": [ { "topic_id": 12, "topic": "Newton's Laws",
               "concepts": [ { "id": 44, "name": "Free-body diagrams", "state": "shaky" } ] } ] }
```

state: **solid** | **shaky** | **untested**.

No aggregate score and no time-on-task field exists anywhere in this response. That omission is the anti-surveillance stance - do not add one.

Practice - student-005

Method	Path	Notes
POST	/student/practice/generate	{ gap_id, count? } -> { practice_set_id, items: [PracticeItem] }
POST	/student/practice/{practice_set_id}/answer	{ item_id, answer }

Misconception check - student-006

The answer response carries the diagnosis when the answer is wrong:

```
{
  "correct": false,
  "correct_answer": "12 N",
  "explanation": "...",
  "citations": [ /* Citation */ ],
  "diagnosis": {
    "id": 91,
    "misconception_id": 7,
    "label": "Treats velocity as implying a net force",
    "question": "It looks like you assumed constant velocity means there's a net force. Does that match your thinking?"
  }
}
```

diagnosis is **null** when the answer is correct or no known error pattern matches.

POST /student/misconception-diagnosis/{diagnosis_id}/confirm - body { "confirmed": true } -> 204.

Only confirmed: true feeds the teacher heatmap. Denied diagnoses are stored but excluded from every teacher

aggregate.

Tutor - rag-003, rag-004, i18n-001

Method	Path	Notes
POST	/tutor/ask	{ question, language?, topic_id? } -> TutorResponse
POST	/tutor/ask/stream	SSE, polish only - build after everything else

Branch on **outcome**: **answered** - **insufficient_evidence** - **graded_work_refused**. All three are **200**, not errors - a refusal is a successful, correct response.

SSE frames, if built: **event**: **token** with **data**: {"text": "..."}, then a final **event**: **done** carrying the complete **TutorResponse** (the alignment badge can only render after the stream ends, since the evidence check needs the full answer).

Assigned reteach

GET /student/assignments -> { items: [{ id, title, body, assigned_at, citations }] }. Only teacher-approved units appear here.

Teacher

Every teacher response is anonymized: no student_id, no name, no email, in any field. Verify this in the API payload, not just the rendered UI.

Misconception heatmap - teacher-001

GET /teacher/misconceptions/heatmap?course_id=3&topic_id=12

```
{ "topic": "Newton's Laws", "class_size": 40, "updated_at": "2026-08-23T12:02:00Z",  
  "items": [ { "misconception_id": 7, "label": "Treats velocity as implying a net force",  
               "confirmed_count": 14, "share": 0.35, "problem_type": "newtons-second-law" } ] }
```

Ranked by **confirmed_count** desc. Frontend **polls** this (suggested 5s while the demo is live).

Reasoning-path breakdown - teacher-002

GET /teacher/problems/{problem_type}/reasoning-paths

```
{ "items": [ { "misconception_id": 7, "label": "...", "confirmed_count": 14,  
               "example": { "given_answer": "24 N", "reasoning": "..."} } ] }
```

Prerequisite gap map - teacher-003

GET /teacher/gap-map?course_id=3&topic_id=12

```
{ "items": [ { "concept": "Vector components", "prerequisite_course": "Class 12 Physics",  
               "students_missing": 11, "share": 0.275 } ] }
```

Uncertainty flags - teacher-004

Method	Path	Notes
GET	/teacher/uncertainty-flags	?status=open
POST	/teacher/uncertainty-flags/{id}/resolve	{ note? } -> 204

```
{ "id": 55, "question": "Explain Lagrangian mechanics", "alignment_percent": 11,
  "reason": "no_matching_material", "topic_id": 12, "occurred_at": "...", "status": "open" }
```

Rows are written automatically by rag-003. No separate wiring.

Before/after tracking - teacher-005

GET /teacher/misconceptions/{misconception_id}/before-after

```
{ "misconception_id": 7, "label": "...",
  "before": { "window": "...", "confirmed_count": 14, "share": 0.35 },
  "after": { "window": "...", "confirmed_count": 5, "share": 0.125 },
  "reteach_at": "2026-08-23T10:00:00Z", "delta_share": -0.225 }
```

after is null when no reteach has happened yet.

Auto-suggested reteach - teacher-006

Method	Path	Notes
POST	/teacher/reteach/suggest	{ misconception_id } -> ReteachUnit (status: "draft")
PATCH	/teacher/reteach/{id}	{ title?, body? }
POST	/teacher/reteach/{id}/approve	-> status: "assigned"

```
// ReteachUnit
{ "id": 12, "misconception_id": 7, "title": "...", "body": "...",
  "practice_items": [ /* PracticeItem */ ], "citations": [ /* Citation */ ],
  "status": "draft", "approved_by": null }
```

A draft unit is never visible at GET /student/assignments. The approval gate is the human-in-the-loop story - never auto-assign.

AI-sourced content verification queue - teacher-007

Method	Path	Notes
GET	/teacher/verification-queue	?status=pending
POST	/teacher/verification-queue/{id}/approve	-> status: "approved"
POST	/teacher/verification-queue/{id}/reject	{ reason? }

```
{ "id": 3, "source_url": "https://...", "title": "...", "excerpt": "...",
  "found_for_gap": "Lagrangian mechanics", "status": "pending", "found_at": "..." }
```

pending items must be unreachable from every student endpoint. Items are seeded for this build - no live web search is implemented.

Features with no HTTP surface

Documented here so nobody goes looking for an endpoint that was never meant to exist.

Feature	Where it lives
infra-001 shared database	.env DATABASE_URL + backend/scripts/check_db.py
infra-003 this contract	this file
infra-005 tunnel	cloudflared + VITE_API_BASE in frontend/.env.local
a11y-001 accessibility	client-side only - see below

auth-002 forgot password

client-side only, deliberately no route

demo-001 rehearsal

docs/demo-script.md

Accessibility - a11y-001

No endpoints. Read-aloud uses the browser's `speechSynthesis` on `TutorResponse.body`; font size and high contrast are client-side state. The only server involvement is `preferred_language` on `User`.

Language handling - i18n-001

Send `language` on `/tutor/ask` and `/student/gaps/{id}/lesson`, or rely on `User.preferred_language`. The server translates in, retrieves in English, answers in English, and translates out. ``citations[]`` **always references the English source book and page**, and `alignment_percent` is computed on the English text - so it does not drift between languages.

What the frontend can mock today

Every shape above is final enough to mock. Suggested order, matching `feature_list.json` priority:

1. `POST /auth/login -> { token, user }`
 2. `GET /student/course-summary`
 3. `GET /student/diagnostic -> POST /student/diagnostic/{id}/submit -> { gaps }`
 4. `GET /student/gaps/{id}/lesson -> TutorResponse` (mock all three `outcome` values)
 5. `POST /student/practice/generate -> POST .../answer -> { diagnosis } -> confirm`
 6. `GET /teacher/misconceptions/heatmap`
-

Change log

When	Change
2026-08-23	Initial draft covering all 32 features.
2026-08-23	<code>/health</code> documented as always-200 with a degraded state; implemented in infra-002.
